

Merkle Trees in Blockchain

Chapter – 7

Fall 2025

Middle Tennessee State University

Use cases of Merkle Tree



Distributed
version control
system



Open-source
peer to peer
distributed
protocol



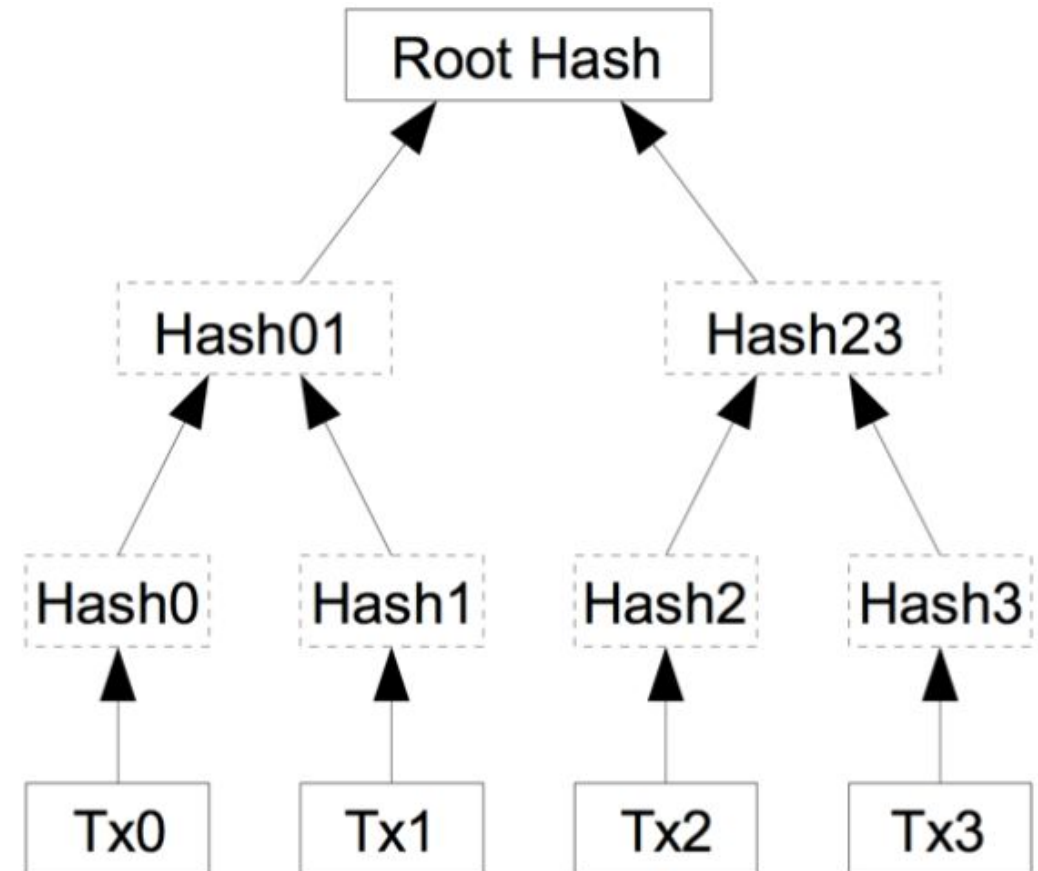
No-SQL
distributed
database



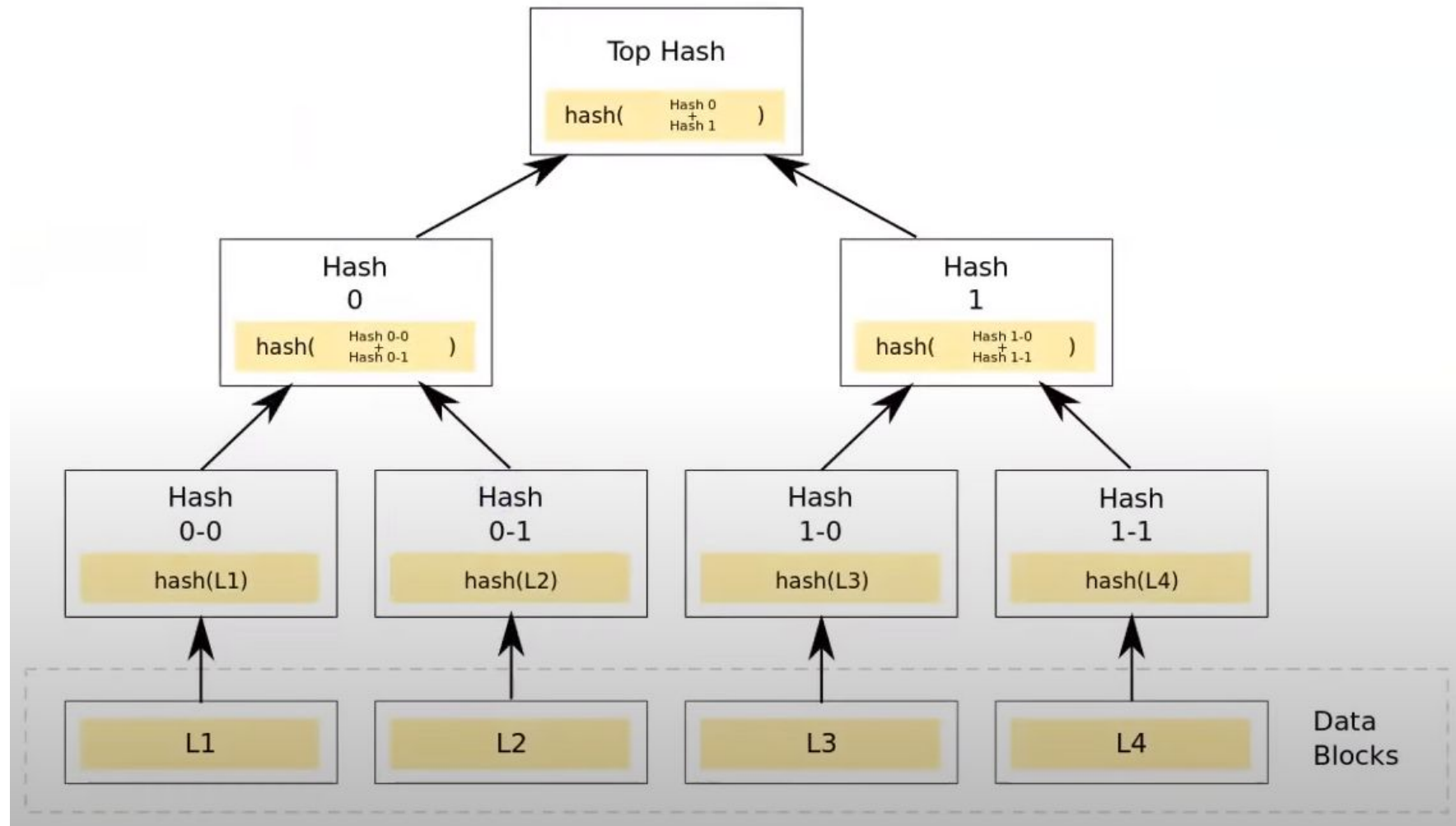
Distributed
Ledger
Technologies,
a.k.a. Blockchain

What is Merkle Tree?

- A type of **data structure**
- Method of organizing data
- Made up of hash number of various data blocks of transactions in blockchain
- **Merkle tree -> “summary” of all the transactions**
- Used to secure the data (by hashing) inside the blockchain effectively
- Also enables quick and secure content verification across big datasets

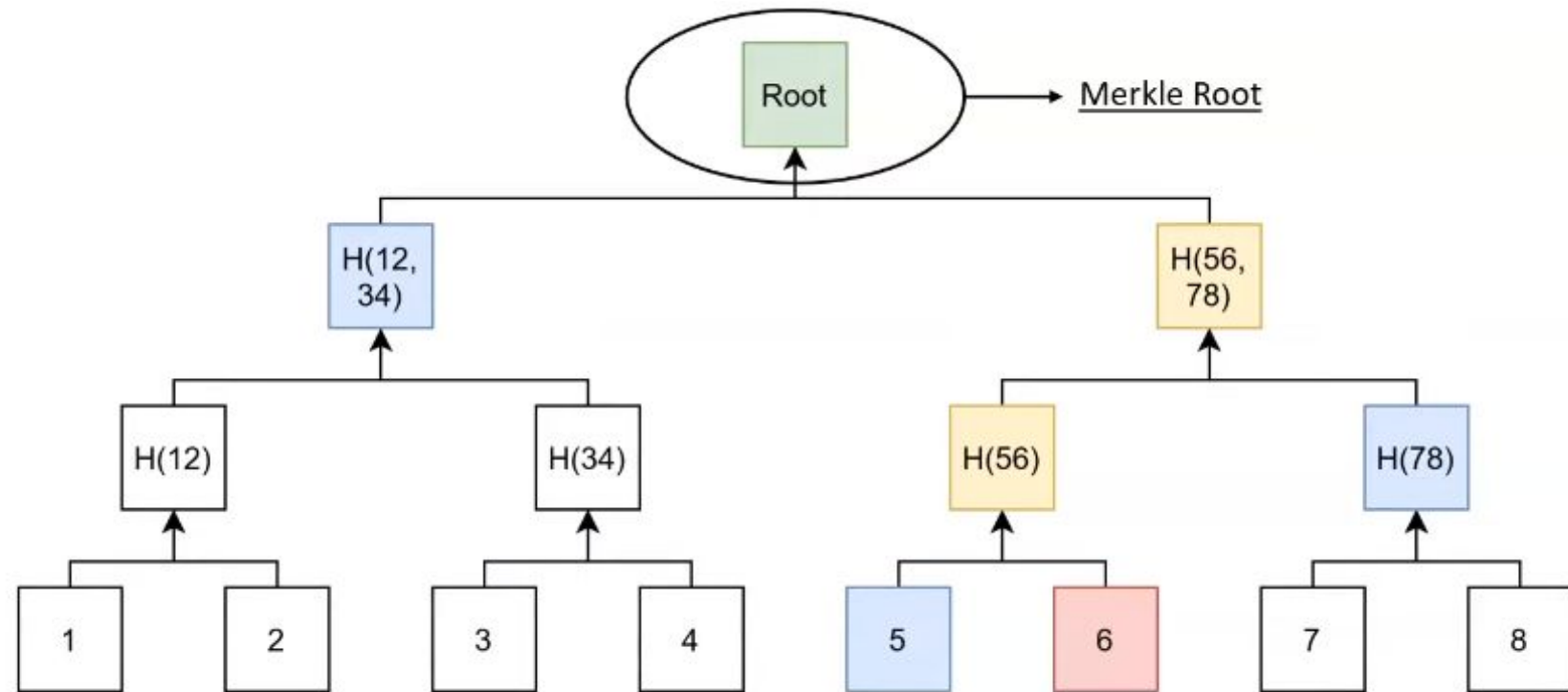


What is Merkle Tree?



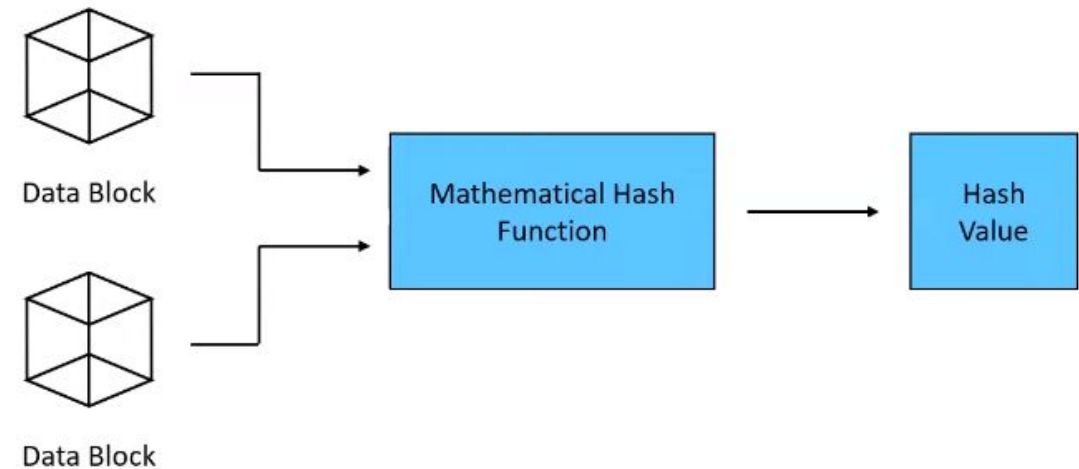
What is Merkle Root?

- Simple mathematical method for **confirming** the facts on a Merkle Tree
- Ensures that data blocks sent through a peer-to-peer network are **secure and consistent**



Cryptographic Hash Functions in Blockchain

- Hash function converts any type of data of any length to a fixed-sized output
- Digital fingerprint of the data
- Irreversible
- Many types of hash functions available
- According to **SHA256** Hash (64 hexadecimal characters) function.
- Ethereum uses Keccak-256
- Hash of 'M' ->
08f271887ce94707da822d5263bae19d5519cb3614e0daedc4c7ce5dab7473f1
- Hash of 'm' ->
62c66a7a5dd70c3146618063c344e531e6d4b59e379808443ce962b3abd63c5a
- Hash of 'MTSU' ->
250cf532fe293e6d93a1c947310d17198e78a2d147d9c9cbab161fda2f30ccfc
- <https://emn178.github.io/online-tools/sha256.html>



Properties of Hash Functions

1. Deterministic

- For the same input, a hash function always produces the same output
- Example: Hashing the string “hello” with SHA-256 will always yield the same hash:
`2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824`

2. Cannot be reverse engineered

- Extremely difficult to reverse a SHA-256 hash to discover the original data due to **one-way function**

3. Collision Resistant

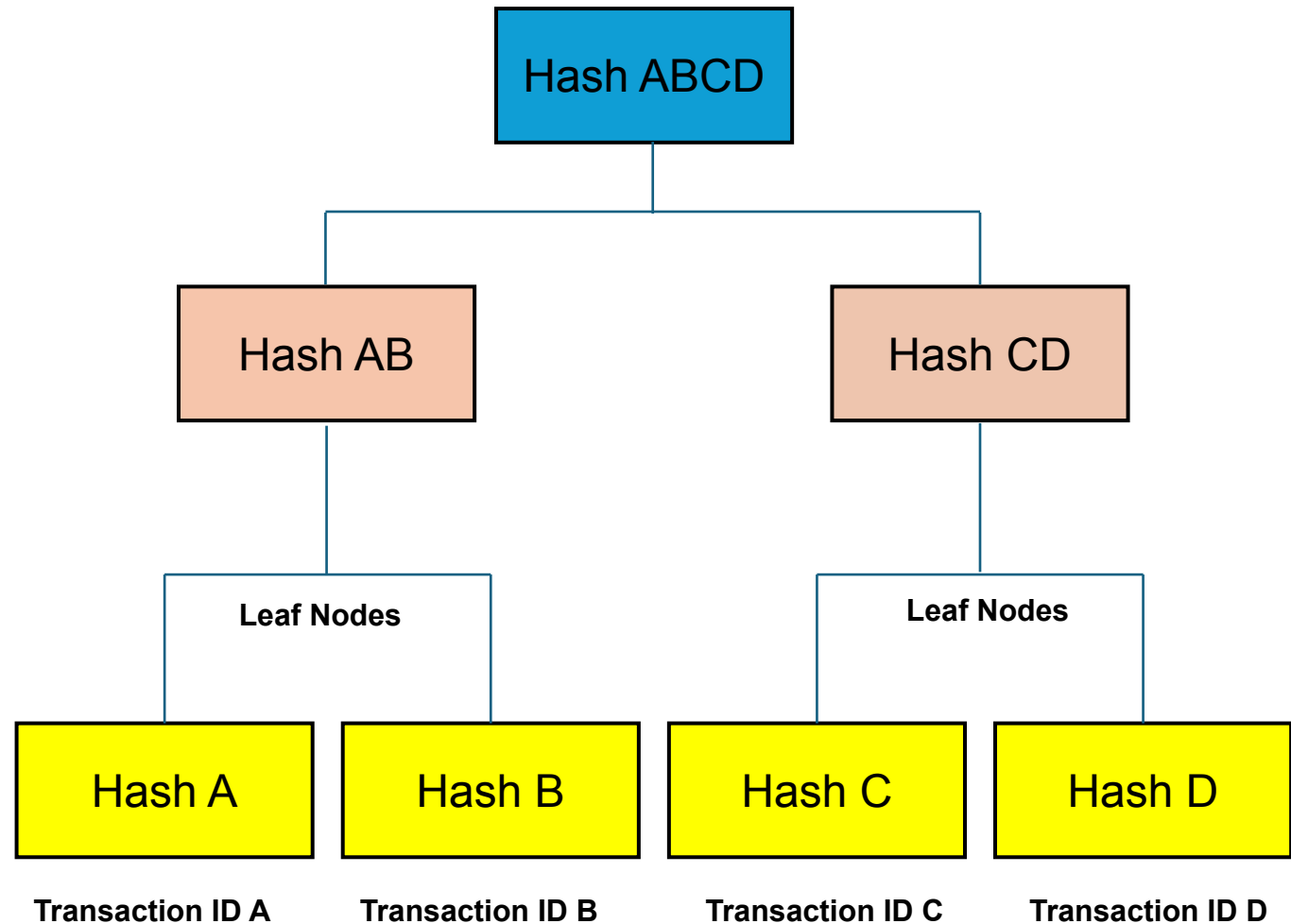
- Highly improbable for two different inputs to produce the same hash value
- Example: Two different person will not have the same fingerprint

4. Computationally Efficient

- Calculating hash is quick and doesn't consume excessive computational resources

Working mechanism of Merkle Tree

- Merkle tree totals all transactions in a block
- Generates a digital fingerprint of the entire set of operations
- Allows users to verify whether a transaction is included in the block
- Merkle trees are built from the bottom using Transaction IDs in blockchain

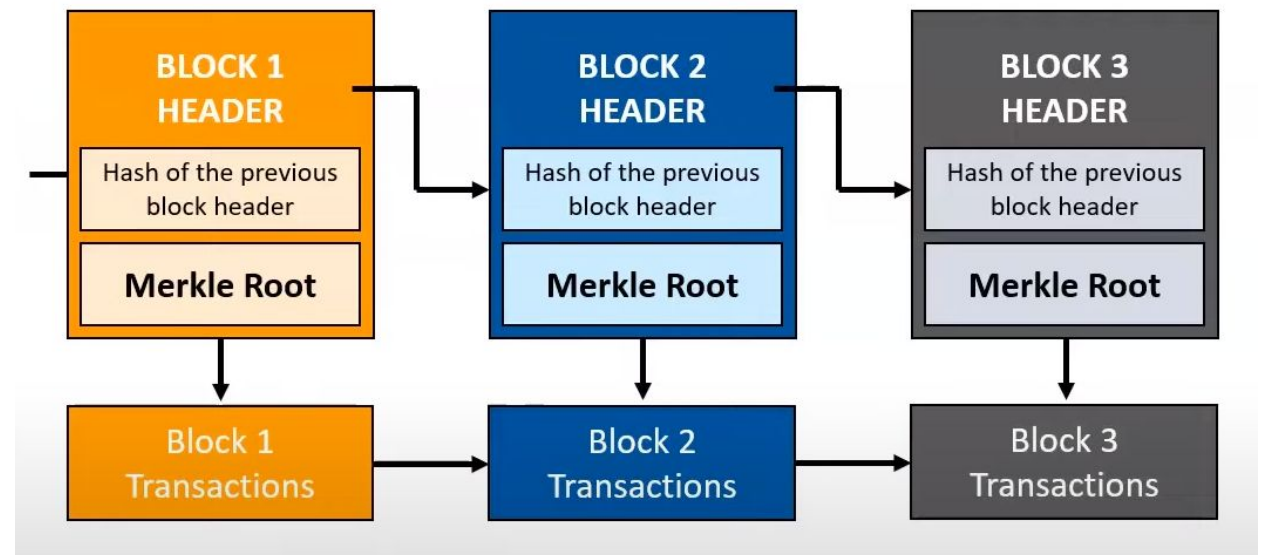
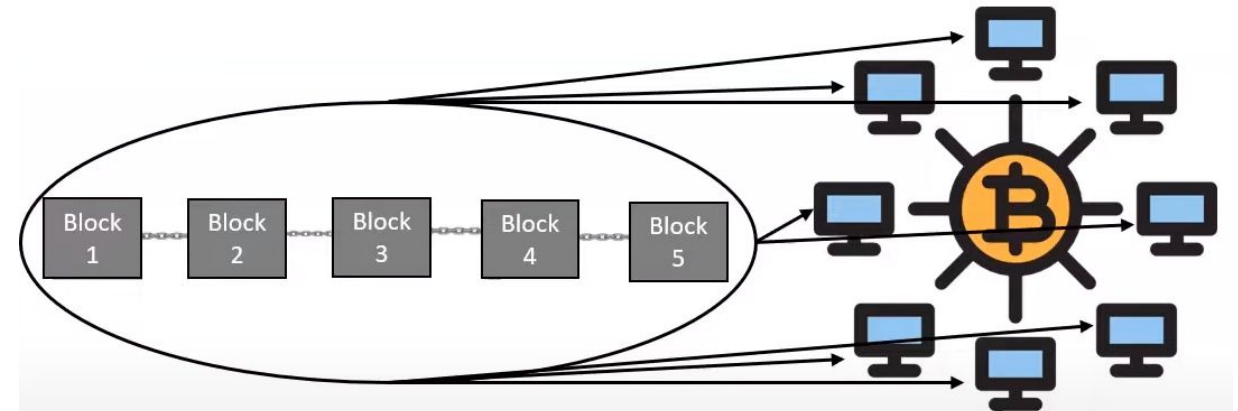


Benefits of Merkle Tree

1. Validate the data's integrity
 - Merkle Trees Provide a tamper-evident structure, ensuring the integrity of data
2. Takes little disk space
 - Merkle Trees are space-efficient since they represent large datasets with a compact Merkle Root
3. Tiny information across network
 - Transmitting a Merkle Root over the network is significantly faster and consumes less bandwidth than sending the entire dataset
4. Efficient verification
 - In a distributed ledger, Merkle Trees facilitate quick validation of a transaction's inclusion in a block without downloading the entire block's data

Why is Merkle Tree crucial to Blockchain?

- Too much information to for every node to validate on their node
- Every node on the network would have to obtain a complete copy of every single transaction ever made
- Would need a lot of computing power to compare the ledgers
- **Which one is bigger? Data vs. Proof of Data**



End of Chapter-7